

SOFTWARE RELEASE NOTE AND TEST EVIDENCE REPORT

Rapid Tech Insights · Rapid Inventory 1.1.0 (proposed) · 29 October 2026

Table of Contents

1. DOCUMENT VERSION CONTROL 2
2. PRODUCT RELEASE NOTE 3
3. ROOT CAUSE ANALYSIS AND FIXES 6
3.1 Inventory overselling (INV-001) 6
3.2 Slow inventory listing (INV-002) 6
3.3 SignalR stock synchronisation (INV-003) 7
3.4 Order placement refactoring (INV-004) 8
3.5 Low-stock alerts (INV-005) 10
4. TEST EVIDENCE 12
4.1 Before and after – inventory SQL 12
4.2 Concurrent orders and final stock 12
4.3 Automated test results 13
4.4 Git commits for the main fixes 13
4.5 Outstanding and known issues 13
5. RELEASE APPROVAL 14

1. DOCUMENT VERSION CONTROL

Date	Version	Author	History	Approver
29/10/2026	1.0	alexkemboi [full name to be confirmed]	First release note and test evidence report (RTI-SRN-RINV-2026-001).	[Pending]
29/10/2026	1.1	alexkemboi [full name to be confirmed]	Rebuilt on the original template with Rapid Tech Insights branding; added root cause analysis, Swagger and frontend screenshots.	[Pending]

2. PRODUCT RELEASE NOTE

Contact Details			
Developer Name	alexkemboi [full name to be confirmed]	**Support Email**	[To be confirmed]
Support Phone No.	[To be confirmed]		
Release Maker	[To be confirmed]	**Release Checker**	[To be confirmed] – Quality Assurance
General Release Info			
Release Date	[To be confirmed] Code complete 29/10/2026	**Release Number**	1.1.0.291020260602 (proposed) (Major.minor.patch.ddmmyyyyhhmn)
Application Name	Rapid Inventory – supermarket inventory and order system	**Application version**	1.1.0 (proposed – the repository has no version number)
Branch Applicability	main – github.com/alexkemboi/Supermarket-Inventory-System	***Client Name***	[To be confirmed]

Fix Release Details	
Source Filename	`rapid-inventory-1.1.0-src-9e622be.zip` (git archive of commit 9e622be, release artifacts excluded)
CRC Check	CRC32: **5E374E3E** SHA-256: `23168f702820…fb31c` Modified 29/10/2026 06:52 (EAT) · 508,998 bytes Values computed from the archive (not a screenshot) – see `EV-G6b_release_package_checksums_9e622be.txt`.
Installation Guide and Environment Setup	The package contains the ASP.NET Core API, the Angular web app, Docker Compose files, tests and seed scripts. 1. **Prerequisites:** Docker Desktop with 6 GB memory; ports 1433, 4200 and 5080 free. 2. **Database setup:** automatic. On start the API applies migrations `InitialCreate` and `AddNonNegativeStockConstraint`, then seeds 2,000 products and 250,000 orders if the database is empty. Back up any existing database first. 3. **Environment setup:** run `docker compose up --build` from the repository root and wait until `db` shows healthy. 4. **Check:** app http://localhost:4200 · API health http://localhost:5080/health · API docs http://localhost:5080/swagger

Change Request	Stage 3 practical brief – five reported issues (INV-001 to INV-005, Section 3). CR reference: [To be confirmed]
CRM Case Number	[To be confirmed]
Purpose of Release/Overview	This release fixes three reported bugs, tidies the order code and adds low-stock alerts: 5. **Bug fix:** concurrent orders could sell more stock than existed. 6. **Performance:** the inventory page was slow (202 database queries per load). 7. **Bug fix:** the live stock board did not update for other users. 8. **Refactor:** order logic moved out of the controller into a service. 9. **Enhancement:** low-stock alerts shown to every connected user.
Requirement	10. Stock can never be oversold or go negative, even with several API servers. 11. A failed order leaves no order records and no stock change. 12. The inventory list returns the same data, without one query per row. 13. Stock changes reach every connected user; failed orders send nothing. 14. An alert fires only when an order takes stock from at/above the reorder level to below it. 15. Existing API responses and error messages stay the same.
Modules/Functions Affected	16. `backend/Services/OrderService.cs` – place, cancel, restock (new home of order logic) 17. `backend/Controllers/OrdersController.cs` – now HTTP only 18. `backend/Controllers/ProductsController.cs` – inventory list query 19. `backend/Services/IStockNotifier.cs` – SignalR broadcast 20. Database: CHECK constraint on `Products.Stock` (migration) 21. `frontend/src/app/stock-realtime.service.ts`, `low-stock-alerts.ts`, app shell 22. `backend/Program.cs` – Swagger UI at `/swagger` (development only)
Test Instruction	23. `dotnet test Inventory.sln` – 42 backend tests on real SQL Server. 24. `npx ng test --watch=false --browsers=ChromeHeadless` (in `frontend/`) – 17 tests. 25. `node seed/two-browser-check.mjs FPR-000021` – two live browsers. 26. `./seed/stress-orders.sh` – 40 concurrent orders; units sold must equal stock taken. Results are in Section 4.
Prerequisite	27. Automated tests pass – **done** (test environment). 28. Independent QA review – **outstanding**. 29. Staging deployment and smoke test – **outstanding**.

	30. Database backup; confirm no product has negative stock before the migration – **outstanding**. 31. Review secrets and configuration (compose uses development values) – **outstanding**. 32. SignalR backplane if more than one API server will run – **outstanding**. 33. Release approval (Section 5) – **outstanding**.
Rollback process	34. Redeploy the previous version of both `api` and `web` (they are compatible either way). 35. **Do not restore a database backup** to undo this release – that would lose real orders. The old code works with the new constraint. 36. Only if needed: remove the constraint with `dotnet ef database update 20261023081746_InitialCreate` (no data is lost). 37. Check: db healthy, `/health` OK, one test order works, `SELECT COUNT(*) FROM Products WHERE Stock < 0` returns 0. Rollback has not been rehearsed yet.

3. ROOT CAUSE ANALYSIS AND FIXES

Each issue was reproduced on the original code (commit `fc3c873`) before it was fixed. All tests ran against real SQL Server.

3.1 Inventory overselling (INV-001)

Problem: Customers could order more units than were in stock. On the original code, 40 orders were all accepted for a product with 10 in stock.

Root cause: The old code read the stock, checked it in memory, then wrote a new number with `SET Stock = @newStock`. Two orders arriving together both read the same value and overwrote each other's update. The stock change and the order were also saved separately, so a failed order could still take stock.

Solution: Check and deduct in **one statement** – `UPDATE … SET Stock = Stock - @qty WHERE Id = @id AND Stock >= @qty` – inside one transaction with the order (`OrderService.PlaceOrderAsync`). If no row changes, the API returns 409 and undoes everything. A database rule (`CK_Products_Stock_NonNegative`) blocks negative stock. Live updates are sent only after the order is saved.

Verification: Two API instances racing for the last unit: original code **both succeeded**; fixed code **one success, one 409**, stock 0. 40 orders against 10 in stock: exactly 10 accepted. Two real API processes: 41 + 39 = 80 sold from 80. Stress script: 29 in stock, exactly 29 sold. (`OrderConcurrencyTests`, 12 tests pass.)



Figure 1: *Insufficient-stock order (quantity 9999 for product 81) rejected by the backend with 409 Conflict (Swagger UI).*

3.2 Slow inventory listing (INV-002)

Problem: The inventory page was slow to load and search.

Root cause: The page loads 200 products. The old code fetched them, then ran **one extra query per product** to get its category name – 202 queries per page load. The 250,000 orders were not involved.

Solution: One query that joins products to categories and selects only the columns the page needs (`ProductsController.List`). No index was needed – the plan already reads the primary key in order.

Verification: SQL log: 202 → 2 queries. Typical response 225 ms → 12 ms on the same data, with an identical response (28,978 bytes). `ProductListTests` (4) pass.

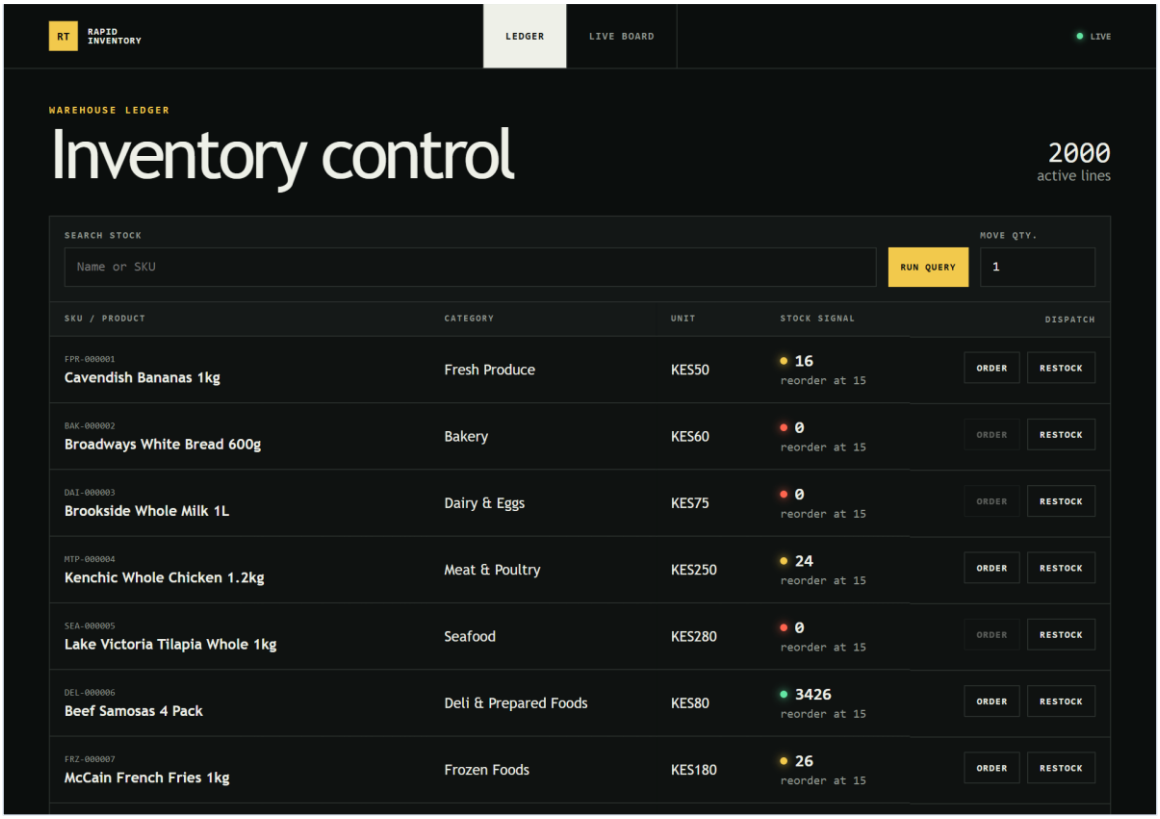


Figure 2: *Inventory list displaying current product quantities (Angular, live connection).*

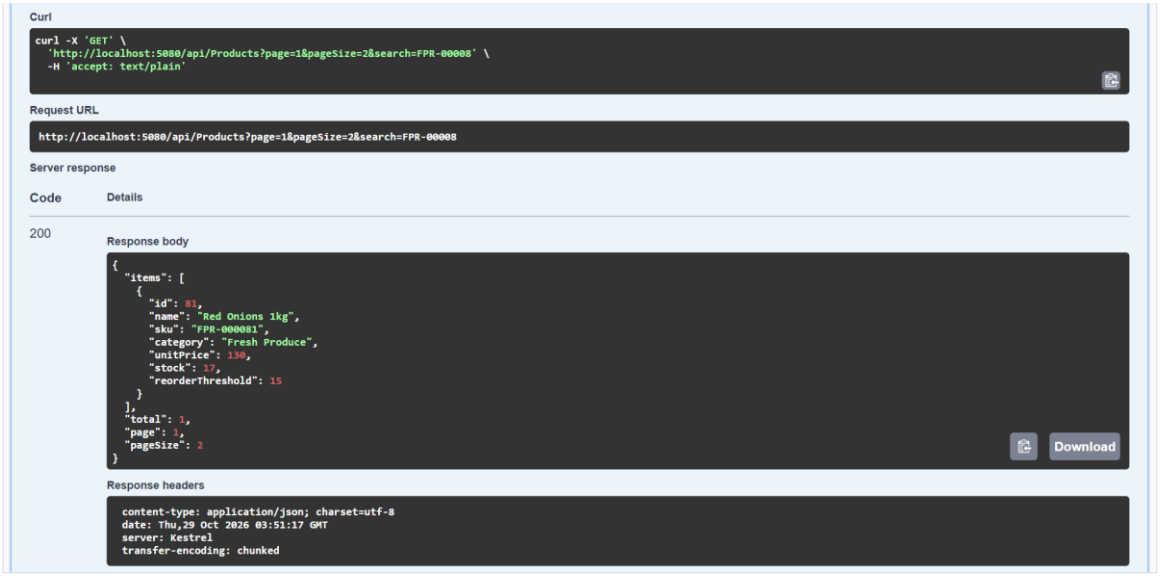


Figure 3: *Inventory retrieval endpoint GET /api/Products returning 200 with product, category and stock.*

3.3 SignalR stock synchronisation (INV-003)

Problem: When one user placed an order, other users' screens kept showing the old stock.

Root cause: The hub, URLs and event names were correct. The bug was who received the message: if a connection ID was sent with the order, `SignalRStockNotifier` sent the update **only to that connection**. The inventory page always sends one, so only the buyer saw the change.

Solution: Always send `StockUpdated` to the `stock-watchers` group (`IStockNotifier.cs`). The Angular service also keeps retrying if the connection drops, and reloads data when it is back.

Verification: Two separate browser sessions: before the fix Browser B stayed at 41 while the database said 40; after the fix Browser B updated about 4 ms after Browser A without a refresh. A failed order sent nothing. 5 SignalR tests pass. Live reconnect after an outage was only unit-tested.

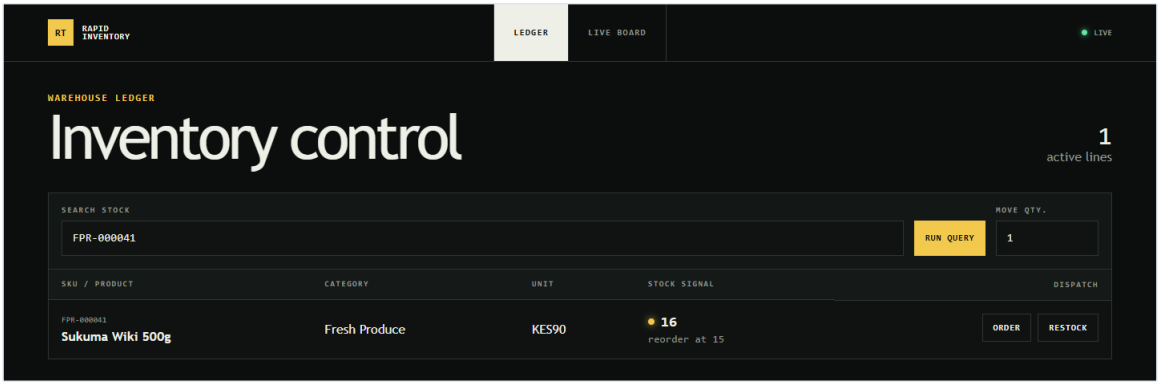


Figure 4: *Successful order placed in Browser A – stock drops from 17 to 16.*

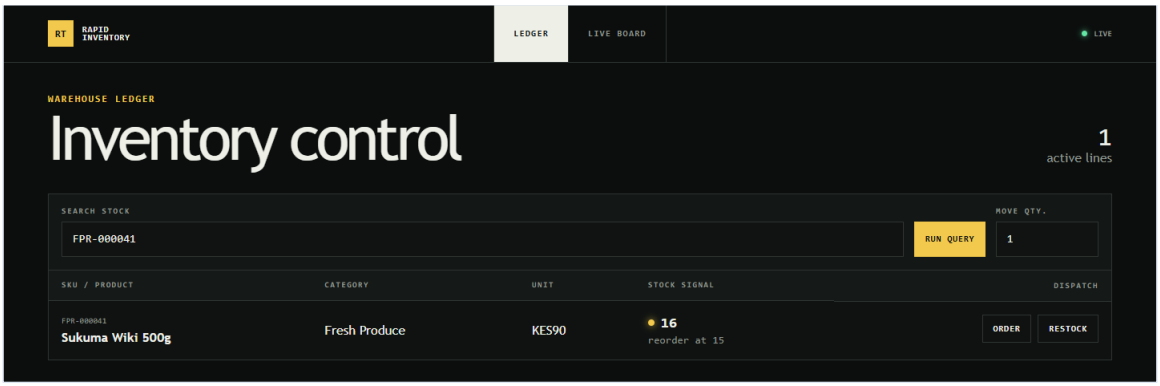


Figure 5: *Stock update received by a second connected browser (Browser B) without refreshing.*

3.4 Order placement refactoring (INV-004)

Problem: `OrdersController` did everything: validation, stock checks, saving, transactions and notifications. An unused `OrderService` held an out-of-date copy of the same logic.

Root cause: Business logic built up in the controller, and the service was never wired in. The cancel action also had a race: 6 of 10 simultaneous cancels of one order each returned the stock.

Solution: `OrderService` now owns placing, cancelling and restocking. The controller only turns results into HTTP responses. Cancel claims the order with a conditional update, so it can only happen once.

Verification: 13 tests that pin the API's status codes, messages and response bodies passed on the old controller and pass unchanged on the new one (`OrderContractTests`). The concurrent cancel test passes.

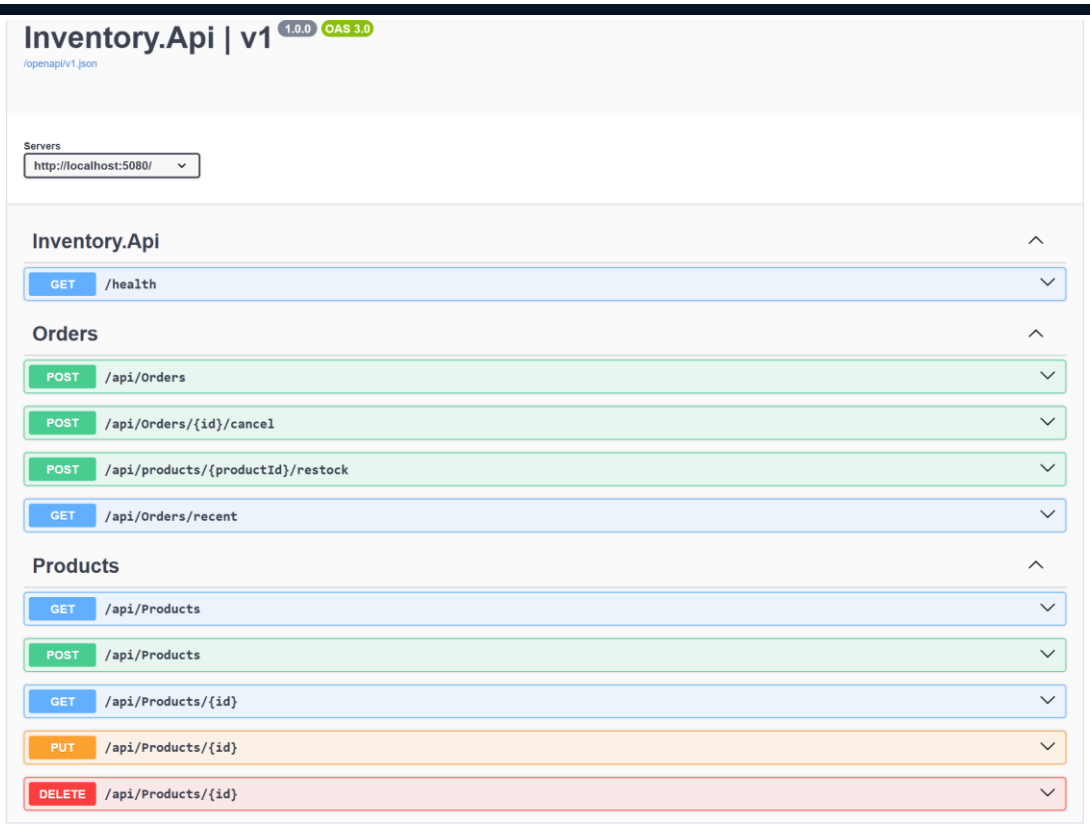


Figure 6: *Swagger UI listing the API endpoints (orders, cancel, restock, recent orders, products, health).*



Figure 7: *API response confirming successful order placement (200 with the new order id).*



Figure 8: *Validation rule unchanged: quantity 0 rejected with 400 and the original error message.*

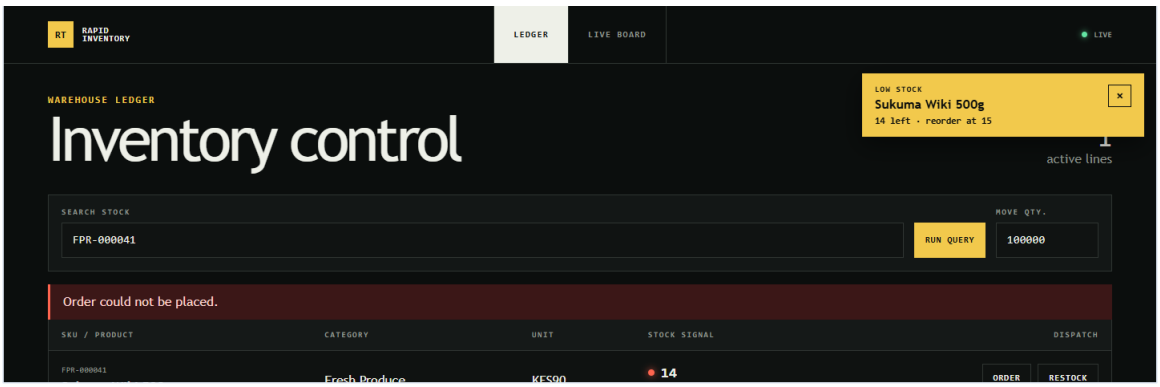


Figure 9: *Error shown in the Angular app when an order cannot be placed (quantity larger than stock).*

3.5 Low-stock alerts (INV-005)

Problem: Users did not see low-stock warnings where they place orders.

Root cause: The backend already sent `LowStockAlert`, but only the Live board page displayed it, and it keyed alerts by product, so a repeat alert for the same product clashed.

Solution: `OrderService` sends `LowStockAlert` (product id, name, stock, reorder threshold) only when an order takes stock from **at or above** the threshold to **below** it, after the order is saved. Angular shows it as a dismissable toast on every page (`low-stock-alerts.ts`), registered once in `StockRealtimeService`.

Verification: 8 rule tests and 17 frontend tests pass. Live, two browsers: at exactly 15 (threshold 15) no alert; at 14 the alert appeared in **both** browsers; a failed order added nothing. Message received: `{"productId":41,"productName":"Sukuma Wiki 500g","stock":14,"reorderThreshold":15}`.

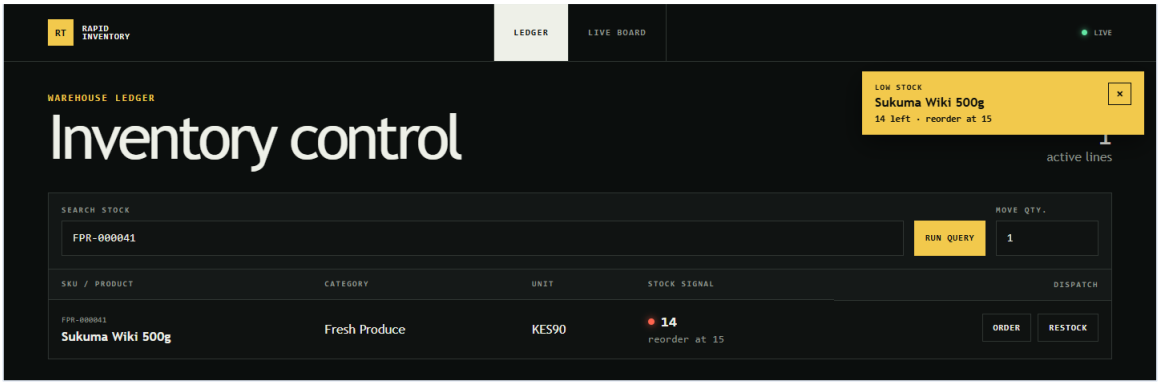


Figure 10: *Low-stock notification displayed in the Angular application (Browser A).*

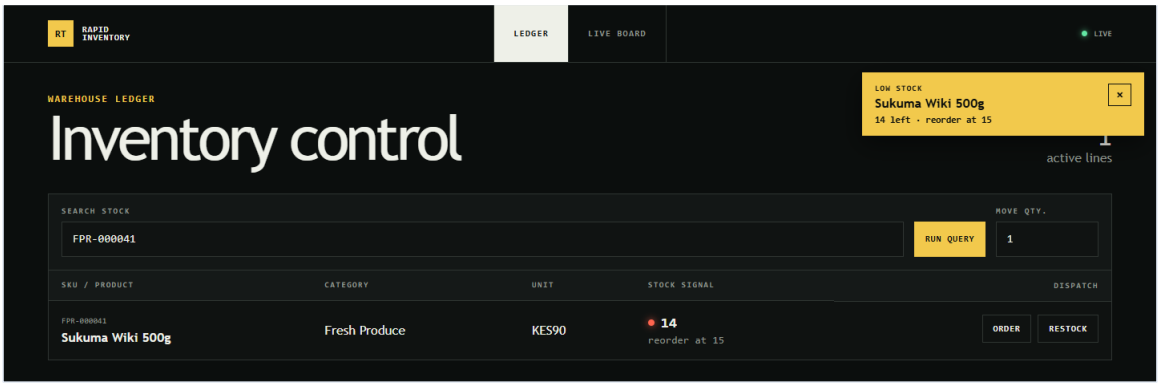


Figure 11: *The same alert delivered to the second connected browser (Browser B).*

4. TEST EVIDENCE

All tests were run on 29/10/2026 on the local Docker Compose stack (SQL Server 2022, .NET 9 API, Angular 20, Chrome 156). Raw logs and screenshots are in `release/Release_Evidence/`.

Test Scenario	Expected Result	Actual Result	Status	Evidence
Concurrent orders	Stock cannot be oversold	Last unit: one 200, one 409. 40 orders / 10 stock: 10 accepted, stock 0. Two live API processes: 80 sold from 80.	PASS	OrderConcurrencyTests; EV-B2, EV-B3; Figure 1
Inventory listing	Correct data with improved query performance	202 → 2 queries; 225 → 12 ms; same response; filtering and paging correct.	PASS	ProductListTests; EV-C1–C4; Figures 2–3
SignalR synchronisation	Other connected clients receive stock updates	Browser B updated without refresh; failed order sent nothing.	PASS	StockHubTests; Figures 4–5
Order placement	Existing business rules remain intact	13 contract tests identical before and after; Swagger shows 200 / 400 / 409 as before.	PASS	OrderContractTests; Figures 6–9
Low-stock alert	Alert appears when stock falls below the threshold	None at 15; toast in both browsers at 14; none after a failed order.	PASS	LowStockAlertTests; Figures 10–11
Reconnect after API outage	Client reconnects and reloads	Unit tests only; live outage not tested.	PARTIAL	Karma specs
Staging deployment, manual browser test, load test on a production build	Run before production	Not executed.	NOT RUN	—

4.1 Before and after – inventory SQL

Captured from the API's own EF Core log for one inventory page load. Before: 202 queries. After: 2.

```
-- BEFORE (repeated 200 times, once per product)
SELECT TOP(2) [c].[Name] FROM [Categories] AS [c] WHERE [c].[Id] = @id
-- AFTER (one query for the whole page)
SELECT p.Id, p.Name, p.Sku, c.Name, p.UnitPrice, p.Stock, p.ReorderThreshold
FROM (SELECT ... FROM Products ORDER BY Id OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY) p
INNER JOIN Categories c ON p.CategoryId = c.Id
```

4.2 Concurrent orders and final stock

The same tests and the repository's stress script, run on the original code and on the fix. The database counts are the source of truth.

Check	Original code	After the fix
Two instances race for the last unit	OK, OK (both sold)	OK, Conflict

Check	Original code	After the fix
40 orders, 10 in stock	40 accepted, final stock 5	10 accepted, 30 refused, final stock 0
Stress script, 40 orders	Stock 30 → 13 while 40 units were sold	Stock 29 → 0, exactly 29 units sold
Negative stock rows in the database	—	0 (CHECK constraint in place)

4.3 Automated test results

38. Backend (`dotnet test`): **42 / 42 passed**. On the original code 31 passed and 9 failed – the 9 failures were the reported bugs.
39. Frontend (Karma): **17 / 17 passed**.
40. Builds: `dotnet build` 0 warnings, 0 errors; `npm run build` succeeded.

4.4 Git commits for the main fixes

Commit	Change
6157517	Prevent concurrent stock overselling (atomic update, transaction, CHECK constraint)
2f076ee	Remove per-row category queries from the inventory list
d8329cc	Broadcast stock updates to all watching clients
75569d9	Move order logic into OrderService; make cancel race-safe
1bfb08b	Low-stock toasts on every page
6c5bed4	Swagger UI at /swagger (development only)
fc3c873	Original code, untouched (baseline)

4.5 Outstanding and known issues

41. More than one API server needs a SignalR backplane (e.g. Redis); otherwise some users miss updates.
42. Alerts are sent once after saving; if sending fails the alert is lost (logged).
43. No duplicate-order protection if a client retries after a timeout.
44. Not executed: staging deployment, person-at-keyboard two-browser test, live reconnect after an outage, production-build load test.
45. Cosmetic: the low-stock toast overlaps the page-heading counter.

5. RELEASE APPROVAL

Role	Name	Decision	Signature	Date
Developer / Release Maker	[To be confirmed]	[Pending]	[Pending]	[Pending]
QA Reviewer	[To be confirmed]	[Pending]	[Pending]	[Pending]
Technical Approver	[To be confirmed]	[Pending]	[Pending]	[Pending]
Business Owner	[To be confirmed]	[Pending]	[Pending]	[Pending]

Recommendation: **Ready subject to outstanding conditions.** All five issues are fixed and verified in the test environment and no test failed.

Before production: QA review, staging deployment, database backup, configuration review, approval above – and a SignalR backplane if more than one API server will run. This document does not approve deployment.